

Dissertation Description: Standard ML for Platforms

Ja Ying Lew (230167106)

October 13, 2025

Supervisor:Gergely Buday

Module Code: [Your Module Code]

Due: Friday 17th October 2025

1 Introduction

1.1 Background

The automatic generation of software artifacts from structured specifications is a fundamental challenge in modern software development. API standardization through specifications such as OpenAPI (formerly Swagger) has enabled the formal definition of RESTful services in a machine-readable format. However, translating these specifications into deployable client libraries, server stubs, and supporting documentation remains a labor-intensive process that typically requires manual intervention and repetition across different target platforms and languages.

The problem of mass generation arises particularly when organisations maintain multiple APIs or versioned endpoints, each requiring corresponding client libraries, documentation, and configuration files across different programming languages and platforms. Manual creation of these artifacts is error-prone, time-consuming, and difficult to maintain as APIs evolve. Template-based code generation offers a systematic approach to this problem, allowing developers to define the structure of generated code once and apply it consistently across multiple data sources.

Mustache is a logic-less templating language that has gained significant adoption in the code generation community. It is described as a language-agnostic templating system that works by expanding tags in a template using values provided in a hash or object structure [1]. The term “logic-less” refers to the absence of explicit control flow statements such as if statements or for loops; however, conditional evaluation and iteration can still be achieved through section tags that process lists and anonymous functions.

1.2 Project Description

This project investigates the use of Standard ML (SML) as an implementation language for building a templating platform capable of generating multiple software artifacts from OpenAPI specifications. Specifically, the work will focus on developing an SML-based system that can parse a JSON representation of an API specification (such as Swagger/OpenAPI) and apply Mustache templates to generate standardized SDK code, configuration files, and supporting documentation.

The project aims to address two interconnected challenges: (1) mass generation of API client libraries and related artifacts from a single specification source, and (2) the design and implementation of a robust templating system that can handle multiple template types and target languages. Rather than adopting a generic templating solution, the project will explore how SML’s strong type system, pattern matching capabilities, and functional programming paradigm can provide advantages in template parsing, validation, and code generation.

The specific use case involves creating a platform that can accept a Swagger/OpenAPI JSON specification as input, parse and structure the API metadata, and then systematically apply Mustache templates to extract relevant information and generate output artifacts. This approach enables developers to maintain a single authoritative specification and derive multiple implementation artifacts without manual duplication or inconsistency.

1.3 Literature Review

Several existing approaches address the problem of automated artifact generation from API specifications. Swagger Codegen and OpenAPI Generator are mature, template-driven tools that automate the generation of API client libraries (SDKs), server stubs, and documentation directly from OpenAPI specifications [3, 4]. Both tools use Mustache as their primary templating engine, allowing approximately 90% of customization needs to be addressed through template modification alone [5]. This prevalence of Mustache in production code generation pipelines demonstrates its practical utility and wide adoption across multiple programming languages and platforms.

Beyond Mustache, alternative templating systems offer different trade-offs. Jinja is a Python-based text template language that provides Python-like expressions while maintaining template sandboxing; it is capable of generating any markup or source code [8]. Handlebars extends Mustache with additional features such as custom helpers and template inheritance, while also offering performance advantages through pre-compilation into native functions [7]. Twig, primarily designed for PHP applications, shares similar syntax with Jinja but is specialized for a particular ecosystem.

The choice between “logic-less” templating systems like Mustache and more feature-

rich engines like Jinja or Handlebars represents a key trade-off. Logic-less systems enforce separation of concerns by restricting business logic to the data provider and keeping templates focused on presentation and structure. However, this simplicity can sometimes necessitate additional preparation of input data to avoid template complexity.

2 Analysis

2.1 Problems to Be Solved

The primary problem addressed by this project is the efficient generation of multiple SDK artifacts and supporting files from a single OpenAPI specification. Without systematic templating, API providers must manually create and maintain client libraries for each supported language, leading to inconsistencies, version skew, and high maintenance overhead. The challenge involves several sub-problems:

Data Extraction and Structuring: OpenAPI/Swagger specifications contain rich metadata about endpoints, parameters, request/response schemas, and data types. This information must be parsed from JSON and transformed into a structured representation suitable for template processing. Specifically, extracting the correct information from the Swagger specification and mapping it into template variables requires careful handling of nested structures and optional fields.

Template Application at Scale: Once data is extracted, a templating system must reliably apply multiple templates (possibly targeting different languages or output formats) to generate consistent, syntactically correct code. This requires not only variable substitution but also iteration over collections (such as endpoint lists or parameter arrays) and conditional inclusion of code sections based on data properties.

Maintainability and Extensibility: The system must support modification of templates without requiring changes to the core generation engine. Template modifications should be achievable by domain experts (API designers or platform engineers) who may not be deeply familiar with the implementation language.

Type Safety and Error Handling: The process must handle malformed specifications, missing required fields, and type mismatches gracefully, providing clear diagnostic information to aid debugging.

2.2 Proposed Techniques and Tools

Standard ML as Implementation Language: SML offers several properties that are beneficial for this task. Its strong, static type system can enforce correct handling of API specification structures at compile time. Pattern matching enables elegant handling of different specification versions and optional fields. The functional programming

paradigm, particularly immutability and first-class functions, aligns well with template processing as a series of data transformations.

Mustache Template Engine: Mustache will be implemented or integrated as the templating layer. Its logic-less nature is advantageous for maintaining the separation between data processing (implemented in SML) and presentation logic (in templates). The tool will parse Mustache syntax and apply templates to extracted API data. Mustache’s language-agnostic design means that the same template definitions can theoretically be used across different target languages, reducing duplication.

JSON Processing: A JSON parser and handler will be required to ingest OpenAPI specifications. SML’s strong typing can be leveraged to create strongly-typed representations of the OpenAPI schema, reducing the chance of runtime errors. The selection of an existing JSON library (such as those available in the SML ecosystem) will be evaluated based on robustness and ease of integration.

Alternative Consideration: Comparison with Other Approaches: The project will briefly consider why alternative templating engines (such as Jinja or Handlebars) were not selected. While these engines offer more sophisticated logic capabilities, they would require the implementation of a more complex interpreter and would shift some data processing logic into templates, potentially reducing maintainability. Mustache’s simplicity and near-universal support aligns better with the goal of creating a reusable, portable system.

2.3 Implementation Strategy

The system will follow a pipeline architecture: (1) JSON parsing and type-safe representation of the OpenAPI specification, (2) extraction and transformation of relevant API metadata into a simplified intermediate representation, (3) Mustache template parsing and preparation, and (4) iterative template application to generate output files.

The use of SML’s module system and functors will enable modular design, with separate components for JSON handling, template processing, and file generation. This modularity will facilitate testing and allow future extensions (such as support for additional specification formats or template engines).

3 Plan of Action

4 Plan of Action

- **Weeks 1–3:** SML fundamentals and literature review. Learn Standard ML syntax, type system, pattern matching, and functional programming paradigms. Review

existing code generation tools (Swagger Codegen, OpenAPI Generator), study OpenAPI 3.0 specification structure, and investigate SML JSON libraries and template implementations available in the SML ecosystem.

- **Weeks 4–5:** Problem analysis and design. Analyse the mass generation problem in detail. Define the data structures and types required to represent an OpenAPI specification in SML. Sketch the overall system architecture and module boundaries. Plan the Mustache template syntax support and design the pipeline flow.
- **Weeks 6–8:** Core implementation phase 1. Implement JSON parsing to create strongly-typed OpenAPI representations. Develop basic data extraction and transformation logic. Establish the module structure and begin testing with sample OpenAPI specifications.
- **Weeks 9–11:** Core implementation phase 2. Implement Mustache template parsing and a template engine capable of handling variable substitution, sections (iteration), and inverted sections. Integrate template processing with the data extraction phase. Conduct unit testing on individual components.
- **Weeks 12:** Integration, testing, and semester 1 consolidation. Build end-to-end pipeline, create comprehensive test cases with real OpenAPI specifications, generate sample artifacts (e.g., Python and JavaScript SDK stubs), validate output correctness, and consolidate learning and implementation from semester 1.
- **(Weeks 13+):** Refinement, documentation, comparative analysis, and dissertation write-up. Optimize code performance and maintainability, document the system design and implementation details, compare generated artifacts with those produced by existing tools, discuss advantages and limitations of the SML-based approach, and prepare the final dissertation. **** struggling in the what is happening in semester 2**
- **Weeks 15+:** Dissertation write-up and final refinement.

References

- [1] Mustache. (n.d.). *Mustache Documentation*. Retrieved from <https://mustache.github.io/>
- [2] Mustache.js. (n.d.). *GitHub*. Retrieved from <https://github.com/janl/mustache.js>
- [3] Swagger Codegen. (n.d.). *Swagger Tools Documentation*. Retrieved from <https://swagger.io/tools/swagger-codegen/>
- [4] OpenAPI Generator. (n.d.). *GitHub*. Retrieved from <https://github.com/OpenAPITools/openapi-generator>
- [5] OpenAPI Generator. (n.d.). *OpenAPI Generator Documentation: Templating*. Retrieved from <https://openapi-generator.tech/docs/templating/>
- [6] OpenAPI Specification Version 3.1.0. (n.d.). *Swagger Documentation*. Retrieved from <https://swagger.io/specification/>
- [7] Handlebars. (n.d.). *Handlebars JavaScript Templating*. Retrieved from <https://handlebarsjs.com/>
- [8] Jinja Template Engine. (n.d.). *Wikipedia*. Retrieved from [https://en.wikipedia.org/wiki/Jinja_\(template_engine\)](https://en.wikipedia.org/wiki/Jinja_(template_engine))
- [9] Wikipedia. (n.d.). *Comparison of web template engines*. Retrieved from https://en.wikipedia.org/wiki/Comparison_of_web_template_engines
- [10] Swagger Codegen: Auto-generating Client SDKs from OpenAPI Specifications. (2023, October 1). *Medium*. Retrieved from <https://medium.com/@jolojiuy/swagger-codegen-auto-generating-client-sdks-from-openapi-specifications-e6f680a2f>